C++中std::map容器的5种初始化方法详解

2025-11-20 12:35:07

在C++标准模板库(STL)中，`std::map`是一个关联容器，用于存储键值对(key-value pairs)。本文将详细介绍`std::map`的5种初始化方法。

初始化方法概览

下表总结了`std::map`的5种初始化方式：

初始化方式	适用场景	C++标准要求
默认构造函数	创建空map	C++98
列表初始化	初始化时指定键值对	C++11
范围构造函数	从其他容器或迭代器范围构造	C++98
拷贝构造函数	复制现有map	C++98
移动构造函数	高效转移现有map资源	C++11

默认构造函数

创建空map，后续通过`operator[]`或`insert`方法添加元素：

C/C++ 复制

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    // 默认构造函数创建空map
    std::map<int, std::string> map1;

    // 添加元素
    map1[1] = "One";
    map1[2] = "Two";

    // 输出结果
    for (const auto& [key, value] : map1) {
        std::cout << key << ": " << value << std::endl;
    }
    return 0;
}
```

列表初始化

C++11引入的初始化方式，直接在构造函数中指定键值对：

C/C++ 复制

```
#include <iostream>
#include <map>
#include <string>

int main() {
    // 列表初始化
    std::map<int, std::string> map2 = {
        {1, "Apple"},
        {2, "Banana"},
        {3, "Cherry"}
    };

    // 输出结果
    for (const auto& [key, value] : map2) {
        std::cout << key << ": " << value << std::endl;
    }
    return 0;
}
```

范围构造函数

从其他容器的迭代器范围构造map：

C/C++ 复制

```
#include <iostream>
#include <map>
#include <string>

int main() {
```

```
    // 原始map
    std::map<int, std::string> original = {
        {1, "Alpha"},
        {2, "Beta"}
    };

    // 范围构造函数
    std::map<int, std::string> map3(original.begin(), original.end());

    // 输出结果
    for (const auto& [key, value] : map3) {
        std::cout << key << ": " << value << std::endl;
    }
    return 0;
}
```

拷贝构造函数

创建现有map的副本：

C/C++ 复制

```
#include <iostream>
#include <map>
#include <string>

int main() {
    // 原始map
    std::map<int, std::string> original = {
        {1, "X"},
```

```
        {2, "Y"}
    };

    // 拷贝构造
    std::map<int, std::string> map4(original);

    // 输出结果
    for (const auto& [key, value] : map4) {
        std::cout << key << ": " << value << std::endl;
    }
    return 0;
}
```

移动构造函数

C++11引入的高效资源转移方式：

C/C++ 复制

```
#include <iostream>
#include <map>
#include <string>
#include <utility>

int main() {
    // 原始map
    std::map<int, std::string> original = {
        {1, "Move"},
        {2, "This"}
    };
```

```
    // 移动构造
    std::map<int, std::string> map5 = std::move(original);

    // 输出map5
    std::cout << "Map5 contents:" << std::endl;
    for (const auto& [key, value] : map5) {
        std::cout << key << ": " << value << std::endl;
    }

    // original现在为空
    std::cout << "Original map size: " << original.size() << std::endl;
    return 0;
}
```

完整示例

以下是一个完整示例，展示所有5种初始化方式：

C/C++ 复制

```
#include <iostream>
#include <map>
#include <string>
#include <utility>

int main() {
    // 1. 默认构造函数
    std::map<int, std::string> map1;
    map1[1] = "Default";
```

```
    // 2. 列表初始化
    std::map<int, std::string> map2 = {
        {1, "List"},
        {2, "Init"}
    };

    // 3. 范围构造函数
    std::map<int, std::string> map3(map2.begin(), map2.end());

    // 4. 拷贝构造函数
    std::map<int, std::string> map4(map3);

    // 5. 移动构造函数
    std::map<int, std::string> map5 = std::move(map4);

    // 输出函数
    auto print_map = [](const auto& m, const std::string& name) {
        std::cout << name << " contents:" << std::endl;
        for (const auto& [key, value] : m) {
            std::cout << "  " << key << ": " << value << std::endl;
        }
    };

    print_map(map1, "Map1");
    print_map(map2, "Map2");
    print_map(map3, "Map3");
    print_map(map5, "Map5");

    return 0;
}
```

编译运行方法：

```bash
g++ -std=c++17 map_init_demo.cpp -o map_init
./map_init
```

页面的所有内容均由人工智能模型生成，其生成内容的准确性和完整性无法保证，请仔细甄别